MentorLink: base de datos y autenticación con Supabase

Entregable del Arquitecto IA

Santiago

4 de noviembre de 2026

Resumen

Este documento contiene la base de datos de MentorLink (tablas `perfiles`, `sesiones` y `mensajes`), sus políticas de seguridad RLS, la evidencia de uso de Supabase CLI y la guía para configurar el inicio de sesión con Email/Password y Google.

Elemento	Valor
Herramienta de IA	Claude (Anthropic)
Supabase CLI	3.0.0 (instalada con `npm install supabase --save-dev`)
Migración	Archivo `inicializar_esquema` con marca de tiempo 20261105023810, en `supabase/migrations/`
Validación	Migración aplicada y 16 pruebas de seguridad ejecutadas en PostgreSQL 16 (resultado: 16 de 16 correctas)

1. Prompt de IA utilizado

Texto exacto enviado a la IA:

```
# ROL
Actúa como Arquitecto de Bases de Datos Senior especializado en PostgreSQL y
 ↪  Supabase, con experiencia en seguridad a nivel de fila (Row Level Security) y
 ↪  en proyectos con presupuesto limitado y tiempos de entrega cortos.

# CONTEXTO
Estoy construyendo "MentorLink", una plataforma que conecta mentores expertos en
 ↪  tecnología con estudiantes. El backend es Supabase (PostgreSQL + Supabase
 ↪  Auth). Los usuarios se registran con Email/Password o con Google OAuth, y sus
 ↪  identidades viven en la tabla auth.users.

# TAREA
Genera UN script SQL de migración para
 ↪  supabase/migrations/<timestamp>_inicializar_esquema.sql que cree:
1. perfiles (id, nombre, bio, rol): id es uuid, clave primaria y clave foránea a
 ↪  auth.users(id) con ON DELETE CASCADE. rol solo admite 'mentor' o 'estudiante'
 ↪  (por defecto 'estudiante').
2. sesiones (id, mentor_id, estudiante_id, fecha, estado): claves foráneas a
 ↪  perfiles; estado solo admite 'pendiente', 'confirmada', 'completada' o
 ↪  'cancelada'; mentor y estudiante deben ser personas distintas.
```

```
3. mensajes (id, sesion_id, remitente_id, contenido): claves foráneas a sesiones y
 ↪  perfiles; contenido no vacío y de máximo 2000 caracteres.

# REQUISITOS TÉCNICOS
- Claves primarias uuid con gen_random_uuid() y fechas timestamptz con default
 ↪  now().
- Claves foráneas con ON DELETE explícito, restricciones NOT NULL y CHECK.
- Índices en todas las claves foráneas y en las columnas que usan las políticas.
- Un trigger que cree el perfil automáticamente cuando se registra un usuario en
 ↪  auth.users, tomando el nombre de raw_user_meta_data->>'full_name' (lo envía
 ↪  Google) y, si no existe, la parte local del email.
- Un trigger que valide que mentor_id tenga rol 'mentor' y estudiante_id tenga rol
 ↪  'estudiante'.
- Impedir que un mentor tenga dos sesiones activas a la misma hora.

# SEGURIDAD (RLS)
- Habilita RLS en las tres tablas y revoca todo acceso al rol anon.
- perfiles: lectura para usuarios autenticados; cada usuario edita solo su propio
 ↪  perfil y solo las columnas nombre y bio (no puede cambiar su rol).
- sesiones: un usuario solo lee y actualiza las sesiones donde es mentor_id o
 ↪  estudiante_id; solo el estudiante crea sesiones a su nombre y en estado
 ↪  'pendiente'; sin DELETE.
- mensajes: un usuario solo lee los mensajes de las sesiones en las que participa y
 ↪  solo envía mensajes como sí mismo (remitente_id = auth.uid()) dentro de esas
 ↪  sesiones; sin UPDATE ni DELETE.
- Usa (select auth.uid()) en las políticas, TO authenticated y nombres de política
 ↪  descriptivos en español.
- Toda función SECURITY DEFINER debe fijar search_path = ''.

# FORMATO DE SALIDA
1. Un único bloque de código SQL, organizado en secciones comentadas: Tablas,
 ↪  Índices, Funciones y Triggers, RLS.
2. Debajo, una tabla con las columnas: Tabla | Operación | Quién puede.
3. Una lista de casos de prueba (usuario, acción, resultado esperado) para
 ↪  verificar las políticas.
No uses extensiones adicionales ni funciones que requieran un plan de pago.
```

2. Scripts SQL generados

2.1 Decisiones de diseño

Decisión	Motivo
`perfiles.id` es a la vez clave primaria y clave foránea a `auth.users(id)`	Cada cuenta de Supabase Auth tiene exactamente un perfil. Si se borra la cuenta, el perfil se borra con ella.
Un trigger crea el perfil al registrarse	Funciona igual con Email/Password y con Google, sin código extra en el frontend.
Todo usuario nuevo es `estudiante`	Evita que alguien se registre como mentor por su cuenta. Solo un administrador asigna el rol `mentor`.
Restricciones `CHECK` en `rol`, `estado` y `contenido`	Los datos inválidos se rechazan en la base de datos y no dependen de la validación del frontend.

Decisión	Motivo
Índice único parcial `(mentor_id, fecha)`	Un mentor no puede tener dos sesiones activas a la misma hora. Las canceladas no cuentan.
Sin DELETE en sesiones ni UPDATE/DELETE en mensajes	Las sesiones se cancelan, no se borran, y los mensajes quedan como historial que no se puede alterar.
`(select auth.uid())` en las políticas	Práctica recomendada por Supabase para que PostgreSQL evalúe la función una sola vez por consulta y no una vez por fila.

2.2 Script completo (tablas, índices, triggers y RLS)

```sql
-- ============================================================================
-- MentorLink · Migración: inicializar_esquema
-- Tablas: perfiles, sesiones, mensajes + RLS
-- ============================================================================

-- ----------------------------------------------------------------------------
-- 1. TABLAS
-- ----------------------------------------------------------------------------

-- Perfil público de cada usuario. Su id ES el id de auth.users (1 a 1).
create table public.perfiles (
  id         uuid primary key references auth.users (id) on delete cascade,
  nombre     text not null check (char_length(trim(nombre)) between 1 and 100),
  bio        text check (char_length(bio) <= 1000),
  rol        text not null default 'estudiante'
             check (rol in ('mentor', 'estudiante')),
  creado_en  timestamptz not null default now()
);

-- Sesión de mentoría entre un mentor y un estudiante.
create table public.sesiones (
  id             uuid primary key default gen_random_uuid(),
  mentor_id      uuid not null references public.perfiles (id) on delete cascade,
  estudiante_id  uuid not null references public.perfiles (id) on delete cascade,
  fecha          timestamptz not null,
  estado         text not null default 'pendiente'
                 check (estado in ('pendiente', 'confirmada', 'completada',
                  ↪  'cancelada')),
  creado_en      timestamptz not null default now(),
  constraint sesiones_participantes_distintos check (mentor_id <> estudiante_id)
);

-- Mensajes dentro de una sesión.
create table public.mensajes (
  id            uuid primary key default gen_random_uuid(),
  sesion_id     uuid not null references public.sesiones (id) on delete cascade,
  remitente_id  uuid not null references public.perfiles (id) on delete cascade,
  contenido     text not null check (char_length(trim(contenido)) between 1 and
 ↪  2000),
  enviado_en    timestamptz not null default now()
);
```

```sql
-- ---------------------------------------------------------------------
-- 2. ÍNDICES (claves foráneas y columnas usadas por las políticas RLS)
-- ---------------------------------------------------------------------
create index sesiones_mentor_id_idx      on public.sesiones (mentor_id);
create index sesiones_estudiante_id_idx  on public.sesiones (estudiante_id);
create index mensajes_sesion_id_idx     on public.mensajes (sesion_id, enviado_en);
create index mensajes_remitente_id_idx   on public.mensajes (remitente_id);

-- Un mentor no puede tener dos sesiones activas a la misma hora.
create unique index sesiones_mentor_fecha_unica
  on public.sesiones (mentor_id, fecha)
  where estado <> 'cancelada';

-- ---------------------------------------------------------------------
-- 3. FUNCIONES Y TRIGGERS
-- ---------------------------------------------------------------------

-- Crea el perfil automáticamente al registrarse (Email/Password o Google).
-- Google envía el nombre en raw_user_meta_data ->> 'full_name'.
-- Todo usuario nuevo es 'estudiante'; solo un administrador asigna 'mentor'.
create function public.crear_perfil_nuevo_usuario()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.perfiles (id, nombre)
  values (
    new.id,
    coalesce(
      nullif(trim(new.raw_user_meta_data ->> 'full_name'), ''),
      nullif(trim(new.raw_user_meta_data ->> 'nombre'), ''),
      split_part(new.email, '@', 1)
    )
  );
  return new;
end;
$$;

create trigger al_crear_usuario
  after insert on auth.users
  for each row execute function public.crear_perfil_nuevo_usuario();

-- Garantiza que mentor_id sea un mentor y estudiante_id un estudiante.
create function public.validar_roles_sesion()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  if not exists (select 1 from public.perfiles where id = new.mentor_id and rol =
   ↪  'mentor') then
    raise exception 'mentor_id (%) no corresponde a un perfil con rol mentor',
 ↪  new.mentor_id;
  end if;
```

```
  if not exists (select 1 from public.perfiles where id = new.estudiante_id and rol
   ↪  = 'estudiante') then
    raise exception 'estudiante_id (%) no corresponde a un perfil con rol
 ↪  estudiante', new.estudiante_id;
  end if;
  return new;
end;
$$;

create trigger validar_roles_sesion
  before insert or update of mentor_id, estudiante_id on public.sesiones
  for each row execute function public.validar_roles_sesion();

-- ---------------------------------------------------------------------
-- 4. ROW LEVEL SECURITY
-- ---------------------------------------------------------------------
alter table public.perfiles enable row level security;
alter table public.sesiones enable row level security;
alter table public.mensajes enable row level security;

-- Usuarios anónimos (sin sesión) no tienen ningún acceso.
revoke all on public.perfiles, public.sesiones, public.mensajes from anon;

-- ---------- perfiles ----------
-- Cualquier usuario autenticado puede ver perfiles (para buscar mentores).
create policy "perfiles: lectura para autenticados"
  on public.perfiles for select
  to authenticated
  using (true);

-- Cada usuario solo edita su propio perfil...
create policy "perfiles: editar solo el propio"
  on public.perfiles for update
  to authenticated
  using ((select auth.uid()) = id)
  with check ((select auth.uid()) = id);

-- ...y solo las columnas nombre y bio (no puede auto-asignarse rol mentor).
revoke update on public.perfiles from authenticated;
grant update (nombre, bio) on public.perfiles to authenticated;

-- ---------- sesiones ----------
-- Solo los participantes (mentor o estudiante) ven la sesión.
create policy "sesiones: leer solo las propias"
  on public.sesiones for select
  to authenticated
  using ((select auth.uid()) in (mentor_id, estudiante_id));

-- Un estudiante solo puede solicitar sesiones a su nombre y en estado pendiente.
create policy "sesiones: el estudiante crea sus solicitudes"
  on public.sesiones for insert
  to authenticated
  with check (
    (select auth.uid()) = estudiante_id
    and estado = 'pendiente'
  );
```

```sql
-- Los participantes pueden reprogramar o cambiar el estado.
create policy "sesiones: participantes actualizan"
  on public.sesiones for update
  to authenticated
  using ((select auth.uid()) in (mentor_id, estudiante_id))
  with check ((select auth.uid()) in (mentor_id, estudiante_id));

revoke update on public.sesiones from authenticated;
grant update (fecha, estado) on public.sesiones to authenticated;
-- Sin política DELETE: las sesiones se cancelan (estado = 'cancelada'), no se
 ↪  borran.

-- ---------- mensajes ----------
-- Solo se leen los mensajes de sesiones en las que el usuario participa.
create policy "mensajes: leer solo de mis sesiones"
  on public.mensajes for select
  to authenticated
  using (
    exists (
      select 1 from public.sesiones s
      where s.id = sesion_id
        and (select auth.uid()) in (s.mentor_id, s.estudiante_id)
    )
  );

-- Solo se envían mensajes como uno mismo y dentro de una sesión propia.
create policy "mensajes: enviar como participante"
  on public.mensajes for insert
  to authenticated
  with check (
    (select auth.uid()) = remitente_id
    and exists (
      select 1 from public.sesiones s
      where s.id = sesion_id
        and (select auth.uid()) in (s.mentor_id, s.estudiante_id)
    )
  );
-- Sin políticas UPDATE/DELETE: los mensajes son inmutables (historial auditable).
```

2.3 Resumen de políticas RLS

Tabla	Operación	Quién puede
`perfiles`	SELECT	Cualquier usuario autenticado (para buscar mentores).
`perfiles`	UPDATE	Solo el dueño, y solo las columnas `nombre` y `bio`.
`sesiones`	SELECT / UPDATE	Solo el mentor o el estudiante de esa sesión. Al actualizar, solo se pueden cambiar `fecha` y `estado`.
`sesiones`	INSERT	Solo el estudiante, a su propio nombre y en estado `pendiente`.

Tabla	Operación	Quién puede
`mensajes`	SELECT	Solo los participantes de la sesión a la que pertenece el mensaje.
`mensajes`	INSERT	Solo un participante de la sesión, y siempre como sí mismo (`remitente_id = auth.uid()`).
Todas	Cualquiera	El rol anon (usuario sin iniciar sesión) no tiene acceso.

3. Evidencia de uso de Supabase CLI

3.1 Comandos ejecutados (log real)

```
$ npm install supabase --save-dev
found 0 vulnerabilities

$ npx supabase --version
3.0.0

$ npx supabase init
Finished supabase init.

$ npx supabase migration new inicializar_esquema
⌋
 ↪  {"path":"/home/claude/mentorlink/supabase/migrations/20261105023810_inicializar_esquema.s
 ↪  created"}

$ tree -a supabase
supabase
├── .temp
│   └── cli-latest
├── migrations
│   └── 20261105023810_inicializar_esquema.sql
├── .env.example
└── config.toml

2 directories, 4 files
```

La CLI crea el archivo de migración vacío, con una marca de tiempo en UTC. En ese archivo se pegó el SQL de la sección 2.2. Luego se editó `supabase/config.toml` para habilitar Google (ver sección 4.2).

3.2 Comandos para aplicar los cambios al proyecto en la nube

Paso	Comando	Qué hace
1	`npx supabase login`	Autentica la CLI con tu cuenta de Supabase (abre el navegador o usa `--token`).

Paso	Comando	Qué hace
2	`npx supabase link --project-ref <project-ref>`	Vincula la carpeta local con tu proyecto. El *project ref* aparece en la URL del Dashboard.
3	`npx supabase db push --remote --dry-run`	Muestra qué migraciones se aplicarían, sin ejecutarlas.
4	`npx supabase db push --remote`	Aplica las migraciones pendientes al proyecto vinculado (pide la contraseña de la base de datos).
5	`npx supabase migration list --remote`	Confirma que la migración aparece en el historial del proyecto remoto.

> **Importante sobre Supabase CLI 3.x:** según la ayuda del propio comando (`supabase db push --help`), `db push` y `migration list` trabajan por defecto sobre la base de datos **local**. Para el proyecto en la nube hay que agregar `--remote`. En versiones 2.x se usaba `--linked`.

Para desarrollo local con Docker, el flujo equivalente es `npx supabase start` (levanta Supabase completo en contenedores) y `npx supabase db reset` (recrea la base y aplica todas las migraciones).

3.3 Validación de la migración y de las políticas RLS

En el entorno donde se generó este documento no había acceso a las imágenes Docker de Supabase, así que `supabase start` no podía levantar el stack. Para comprobar el SQL de todas formas:

1. Se levantó un PostgreSQL 16 local.
2. Se simuló lo mínimo de Supabase que usa la migración: los roles `anon`, `authenticated` y `service_role`, la tabla `auth.users` y la función `auth.uid()`, que lee el `sub` del JWT igual que en Supabase.
3. Se aplicó la migración sin errores y se ejecutaron 16 pruebas con tres usuarios: **Ana** (mentora), **Luis** y **Sofía** (estudiantes).

#	Caso	Esperado	Resultado
P1	Registro de 3 usuarios en `auth.users`	Perfiles creados por el trigger	Correcto (“Ana Mentora”, “luis”, “Sofía Estudiante”)
P2	Crear sesión cuyo “mentor” es estudiante	Error del trigger	Correcto
P3	Luis solicita una sesión con Ana a su nombre	Permitido	Correcto
P4	Luis lista sesiones	Ve solo 1 (la suya)	Correcto
P5	Luis envía mensaje en su sesión	Permitido	Correcto
P6	Luis crea una sesión a nombre de Sofía	Bloqueado por RLS	Correcto
P7	Luis escribe en la sesión de Sofía	Bloqueado por RLS	Correcto
P8	Luis envía un mensaje haciéndose pasar por Ana	Bloqueado por RLS	Correcto
P9	Luis se asigna el rol `mentor`	Permiso denegado	Correcto

#	Caso	Esperado	Resultado
P10	Luis edita su bio y luego la de Ana	1 fila y 0 filas	Correcto
P11	Luis borra mensajes	0 filas	Correcto
P12	Sofía lista sesiones y mensajes	1 sesión y 0 mensajes	Correcto
P13	Ana lista sesiones y mensajes	2 sesiones y el mensaje de Luis	Correcto
P14	Doble reserva de Ana a la misma hora	Error de índice único	Correcto
P15	Usuario anónimo lee sesiones	Permiso denegado	Correcto
P16	Mensaje vacío	Error de CHECK	Correcto

Los scripts de prueba y el log completo están en la carpeta `pruebas/` del proyecto.

4. Configuración de autenticación

4.1 Email/Password (Dashboard)

1. En **Authentication**, abre la página de proveedores (*Sign In / Providers*). Verifica que **Email** esté habilitado y activa **Confirm email**, para que nadie se registre con correos ajenos.
2. Sube la longitud mínima de contraseña a **8 caracteres** (Supabase trae 6 por defecto).
3. En **Authentication → URL Configuration**, cambia **Site URL** por la URL de producción de MentorLink (por defecto es `http://localhost:3000`). En **Redirect URLs** agrega las URLs a las que Supabase puede devolver al usuario después de iniciar sesión, por ejemplo `http://localhost:5173/**` en desarrollo.

   La ubicación exacta de algunos ajustes ha cambiado entre versiones del Dashboard. Si no los encuentras donde se indica, búscalos dentro de **Authentication**.

Para el entorno local, lo mismo queda en `supabase/config.toml`:

```
[auth]
site_url = "http://127.0.0.1:3000"
enable_signup = true
minimum_password_length = 8

[auth.email]
enabled = true
enable_confirmations = true

[auth.external.google]
enabled = true
client_id = "env(SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_ID)"
secret = "env(SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_SECRET)"
skip_nonce_check = false
```

El Client ID y el Client Secret se leen de `supabase/.env`, que está en `.gitignore` y nunca se sube al repositorio.

4.2 Guía rápida: habilitar Google en 3 pasos

Paso 1. Copia la Callback URL de Supabase. En el Dashboard ve a **Authentication → Sign In / Providers → Google** y copia la **Callback URL**. Tiene la forma `https://<project-`

ref>.supabase.co/auth/v1/callback. No guardes nada todavía.

Paso 2. Crea las credenciales OAuth en Google Cloud. En Google Cloud Console crea o elige un proyecto y abre **Google Auth Platform**:

- **Branding:** nombre de la app (“MentorLink”) y logo de la pantalla de consentimiento.
- **Audience:** quién puede iniciar sesión (usuarios externos).
- **Data Access:** agrega el scope `openid`. `userinfo.email` y `userinfo.profile` vienen por defecto.
- **Clients → Create client:** tipo **Web application**.
  - En **Authorized JavaScript origins** pon la URL de la app (por ejemplo `https://mentorlink.com`; en desarrollo, `http://localhost:5173`).
  - En **Authorized redirect URIs** pega la Callback URL del paso 1. En local con Supabase CLI se usa `http://127.0.0.1:54321/auth/v1/callback`.
- Pulsa **Create** y guarda el **Client ID** y el **Client Secret**.

Paso 3. Activa el proveedor en Supabase. Vuelve a **Authentication → Sign In / Providers → Google**, activa **Enable Sign in with Google**, pega el **Client ID** y el **Client Secret** y guarda. Prueba desde el frontend:

```
await supabase.auth.signInWithOAuth({
  provider: 'google',
  options: { redirectTo: 'http://localhost:5173/auth/callback' },
})
```

Al primer inicio de sesión, el trigger `al_crear_usuario` crea el perfil usando el nombre que envía Google (`full_name`).

Error frecuente: `redirect_uri_mismatch` indica que la URI registrada en Google no coincide exactamente con la Callback URL de Supabase. Revisa `http` contra `https`, el dominio y la barra final.

5. Limitaciones y siguientes pasos

- **Cambios de estado:** las transiciones de estado no tienen reglas en la base. Cualquier participante puede marcar una sesión como `completada`. Si se necesita, se agrega un trigger que, por ejemplo, solo deje al mentor confirmar.
- **Promover a mentor:** hoy se hace desde el SQL Editor o con la `service_role` key. Si el volumen crece, conviene una función o panel de administración.
- **Docker:** la validación se hizo en PostgreSQL 16 con una simulación de `auth`. Antes de producción conviene correr `supabase start` y `supabase db reset` con Docker, sobre la versión 17 que usa la CLI.

Fuentes

- Supabase Docs: Login with Google
- Supabase Docs: Redirect URLs
- Ayuda integrada de Supabase CLI 3.0.0 (`supabase db push --help`, `supabase link --help`, `supabase migration list --help`)